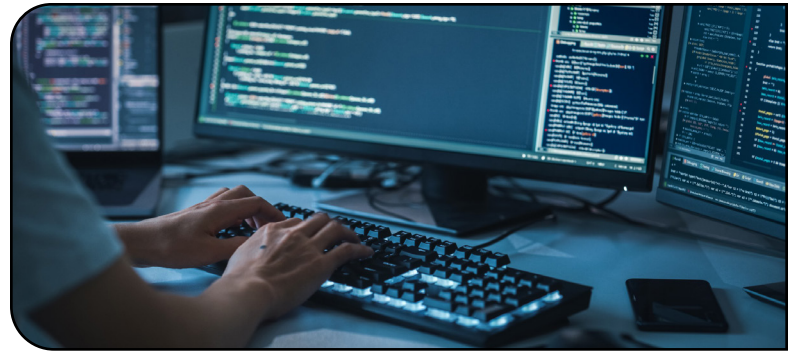


> Usuario

Introduc forms - view - url - template para usuarios ción a sql



```
<> password_reset_confirm.html X
templates > registration > <> password_reset_confirm.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4  <center>
5      <h2>Restablecer contraseña</h2>
6      <form method="POST">
7          {% csrf_token %}
8          {{ form }}
9          <button type="submit">Restablecer contraseña</button>
10     </form>
11 </center>
12 {% endblock %}
```

```
<> password_reset_complete.html ●
templates > registration > <> password_reset_complete.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4  <center>
5      <h2>Contraseña restablecida</h2>
6      <p>Tu contraseña se ha restablecido correctamente.</p>
7  </center>
8  {% endblock %}
```

Hecho esto, solo queda enlazar las urls correspondiente en el template de login, arriba o abajo de registrarse (lo que comunmente se hace) y con eso ya se completaria la parte básica de usuario.
¡Te lo dejamos para que lo hagas!

Es momento de crear los comentarios para los post. Esto se lo puede hacer desde una aplicación nueva o desde una aplicación existente. Para este caso y simplificarlo más, nosotros vamos a usar la aplicación "post" para agregar los comentarios ahí mismo, por lo que en primera instancia vamos a cargar el modelo en models.py de la app "post". Importamos "settings" que vamos a usar para autenticar a los usuarios:

```
apps > posts > models.py > ...
1  from django.db import models
2  from django.utils import timezone
3  from django.conf import settings
4
```

El resto de campo, como el método `__str__` lo basamos en modelos anteriores que creamos.

Lo siguiente que haremos será registrar "Comentario" (previa importación desde el modelo "from .models import Categoria, Post, Comentario") en el administrador de Django desde admin.py:

```
apps > posts > admin.py > ...
16  admin.site.register(Comentario)
```

Creamos el modelo:

```
apps > posts > models.py > ...
35
36  class Comentario(models.Model):
37      posts = models.ForeignKey('posts.Post', on_delete=models.CASCADE, related_name='comentarios')
38      usuario = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='comentarios')
39      texto = models.TextField()
40      fecha = models.DateTimeField(auto_now_add=True)
41
42      def __str__(self):
43          return self.texto
44
```

Donde "posts" es el campo de clave foránea que relaciona cada comentario con un post y si se elimina un post, todos los comentarios relacionados con ese post también se eliminarán debido a la opción `on_delete=models.CASCADE`.

"usuario" es otro campo de clave foránea que relaciona cada comentario con un usuario. Utiliza el modelo de usuario especificado en la configuración `AUTH_USER_MODEL` y establece una relación de "muchos a uno" entre comentarios y usuarios. Cuando se elimina un usuario, todos los comentarios relacionados con ese usuario también se eliminarán debido a la opción `on_delete=models.CASCADE`.

> Comentario

Comentarios sobre los post



Ahora crearemos un nuevo archivo "forms.py" como lo venimos haciendo en las aplicaciones de contacto y usuario, pero ahora lo haremos en la aplicación posts, y lo cargamos:

```
apps > posts > forms.py > ...
1 from django import forms
2 from .models import Comentario
3
4 class ComentarioForm(forms.ModelForm):
5     class Meta:
6         model = Comentario
7         fields = ['texto']
```

Para la vista actualizaremos la vista de "PostDetailView" agregando una función y luego crearemos una nueva vista para comentario (hacemos la importación del form que creamos "from .models import Post, Comentario" y la importación de redirect "from django.shortcuts import redirect"):

```
apps > posts > views.py >
14 class PostDetailView(DetailView):
15     model = Post
16     template_name = "posts/post_individual.html"
17     context_object_name = "posts"
18     pk_url_kwarg = "id"
19     queryset = Post.objects.all()
20
21     def get_context_data(self, **kwargs):
22         context = super().get_context_data(**kwargs)
23         context['form'] = ComentarioForm()
24         context['comentarios'] = Comentario.objects.filter(posts_id=self.kwargs['id'])
25         return context
26
27     def post(self, request, *args, **kwargs):
28         form = ComentarioForm(request.POST)
29         if form.is_valid():
30             comentario = form.save(commit=False)
31             comentario.usuario = request.user
32             comentario.posts_id = self.kwargs['id']
33             comentario.save()
34             return redirect('apps:posts:post_individual', id=self.kwargs['id'])
35         else:
36             context = self.get_context_data(**kwargs)
37             context['form'] = form
38             return self.render_to_response(context)
```

Esta función es un método post que se agregamos a la vista que ya teníamos creada para manejar solicitudes POST. Cuando el usuario envíe el formulario de comentario con solicitud POST al servidor con los datos del formulario, este método procesará esa solicitud y realizará las acciones necesarias para guardar los datos del formulario en la base de datos.

def post(self, request, *args, **kwargs):

>> Crea una instancia del formulario ComentarioForm con los datos enviados en la solicitud POST

form = ComentarioForm(request.POST)

>> Verifica si el formulario es válido (es decir, si todos los campos requeridos están presentes y son válidos)

if form.is_valid():

>> Si el formulario es válido, guarda los datos del formulario en la base de datos
comentario = form.save(commit=False)
comentario.usuario = request.user
comentario.posts_id = self.kwargs['id']
comentario.save()

>> Redirige al usuario a la vista de detalle del post

```
return redirect('apps.posts:post_individual',
id=self.kwargs['id'])
```

else:

>> Si el formulario no es válido, vuelve a mostrar el formulario con los errores de validación

```
context = self.get_context_data(**kwargs)
context['form'] = form
return self.render_to_response(context)
```

Además en el método "get_context_data" agregamos un campo más de contexto para manejar los comentarios que ya están creados y poder mostrarlos:

```
context['comentarios'] = Comentario.objects.filter(posts_id=self.kwargs['id'])
```

Luego creamos la vista de Comentario:

```
apps > posts > views.py > ...
39 class ComentarioCreateView(LoginRequiredMixin, CreateView):
40     model = Comentario
41     form_class = ComentarioForm
42     template_name = 'comentario/agregarComentario.html'
43     success_url = 'comentario/comentarios/'
44
45     def form_valid(self, form):
46         form.instance.usuario = self.request.user
47         form.instance.posts_id = self.kwargs['posts_id']
48         return super().form_valid(form)
```

"ComentarioCreateView" hereda de "LoginRequiredMixin" y "CreateView". "LoginRequiredMixin" es un mixin que requiere que el usuario esté autenticado para acceder a la vista. Si el usuario no está autenticado, será redirigido a la página de inicio de sesión. "CreateView" la hemos visto antes la cual es una vista genérica de Django que proporciona una forma fácil de crear objetos en la base de datos utilizando un formulario (las importaciones son: "from django.contrib.auth.mixins import LoginRequiredMixin" y "from django.views.generic.edit import CreateView").

La vista "ComentarioCreateView" está configurada para utilizar el modelo "Comentario" y el formulario "ComentarioForm". Cuando un usuario accede a esta vista, se mostrará un formulario para crear un nuevo comentario utilizando el formulario "ComentarioForm". Cuando el usuario envía el formulario, los datos del formulario se utilizarán para crear un nuevo objeto "Comentario" en la base de datos.

El método form_valid se utiliza para establecer el usuario y el post asociados con el comentario antes de guardarlo en la base de datos. El usuario se establece en el usuario autenticado actual (self.request.user) y el post se establece en el valor del parámetro "posts_id" pasado en la URL. Una vez que se ha creado el comentario, el usuario es redirigido a la página actual con el comentario creado. Esto lo vamos a incluir en el template de "post_individual.html"

```
templates > posts > post_individual.html > ...
1 {% extends 'base.html' %}
2 {% load static %}
3
4 {% block contenido %}
5 <center>
6 <div class="container-fluid" style="margin: 200px;">
7
8     <table>{{ posts.titulo }}</table>
9     <table>{{ posts.subtitulo }}</table>
10    <table>{{ posts.categoria }}</table>
11    <br>
12    <table>{{ posts.texto }}</table>
13    
14
15 </div>
16 </center>
```



```

templates > posts > post_individual.html > ...
16 </center>
17 <center>
18 <div class="container-fluid" style="margin-bottom: 20px;">
19   <h4>Comentarios</h4>
20   <br><br>
21 </div>
22 <div class="container-fluid" style="margin-bottom: 20px;">
23   {% for comentario in comentarios %}
24     <table>{{ comentario.usuario }} - {{ comentario.fecha }}</table>
25     <ul class="w-100 list-unstyled d-flex justify-content-between mb-0">
26       <p>{{ comentario.texto }}</p>
27       <br><br>
28     </ul>
29   {% empty %}
30     <li>No hay comentarios - Puedes ser el primero en comentar!</li>
31   {% endfor %}
32 </div>
33 <a id="comentario"></a>
34 <div class="container-fluid" style="margin-bottom: 100px;">
35   <form method="POST" style="margin-bottom: 100px; margin-top: 100px;">
36     {% csrf_token %}
37     {% if user.is_authenticated %}
38       <h2>Deja tu comentario</h2>
39       <form method="POST">
40         {% csrf_token %}
41         {{ form.as_p }}
42         <input type="submit" value="Comentar">
43       </form>
44     {% else %}
45       <h2>Debes iniciar sesión o registrarte para comentar</h2>
46       <a class="btn btn-success btn-lg" href="{% url 'apps.usuario:login' %}?next={{ request.path }}#comentario">Iniciar sesión</a>
47       <input type="hidden" name="next" value="{{ request.path }}">
48     {% endif %}
49   </form>
50 </div>
51 </center>
52 {% endblock %}
53
  
```

Por último, antes de poder ejecutar el servidor debemos hacer las migraciones (python manage.py makemigrations y python manage.py migrate) ya que realizamos cambios en el modelo de "post".

Ahora ya tienes las bases para la creación de comentarios. Puedes adaptarlo a tu template para que vaya quedando visiblemente mejor.

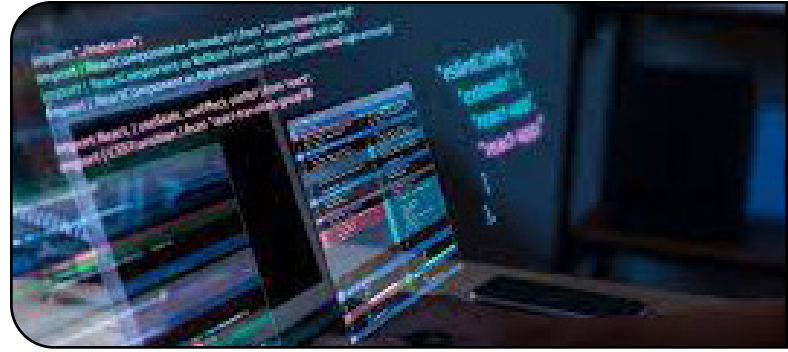
MOSTRANDO RESULTADOS



Hasta aquí llegamos con esta parte del apunte. Próximamente te presentaremos filtros y algunos ajustes más...

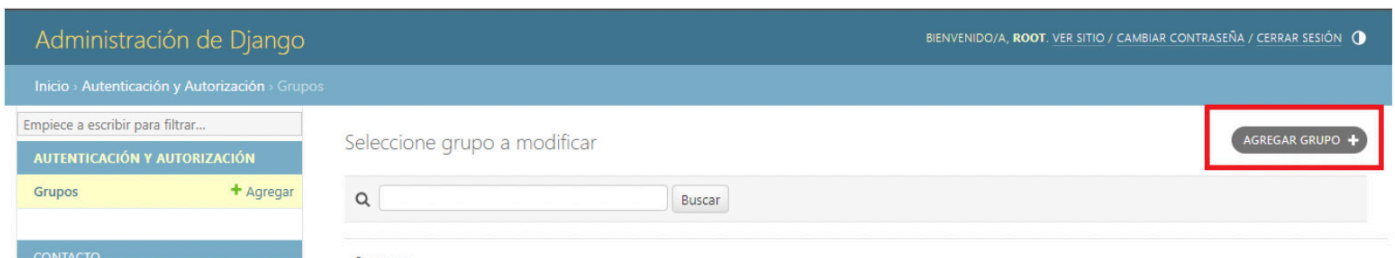
> Permisos de usuarios

Usuario colaborador - usuario registrado

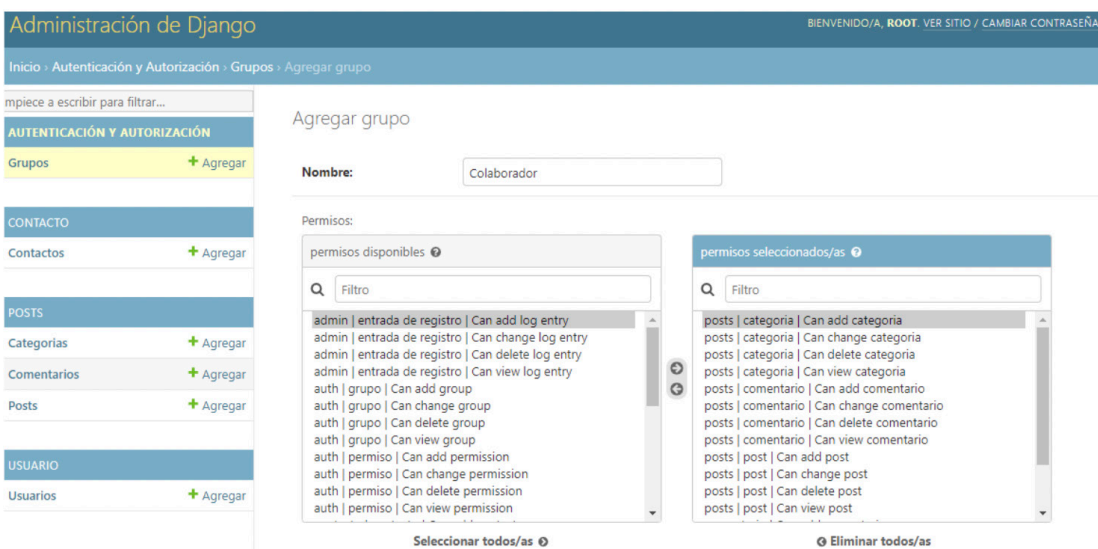


Ahora creamos grupos para identificar a los perfiles de usuarios. En este caso vamos a crear los perfiles de usuario Colaborador y Registrado. Esta es una forma de hacerlo sencilla. También puedes crear estos perfiles desde models.py, pero nosotros usaremos esta forma para que sea más sencillo.

Vamos a dirigirnos al admin de Django donde agregaremos un grupo:



Pondremos el nombre del grupo y le asignaremos los permisos correspondientes. En este caso el usuario Colaborador tendrá todos los permisos para "comentario", "post", "categoria".



Luego crearemos el usuario Registrado el cual tendrá permisos para "comentario".

Inicio > Autenticación y Autorización > Grupos > Agregar grupo

Empiece a escribir para filtrar...

AUTENTICACIÓN Y AUTORIZACIÓN

Grupos + Agregar

CONTACTO

Contactos + Agregar

POSTS

Categorías + Agregar

Comentarios + Agregar

Posts + Agregar

USUARIO

Usuarios + Agregar

Se agregó con éxito grupo "Registrado". Puede agregar otro/a grupo abajo.

Agregar grupo

Nombre:

Permisos:

permisos disponibles

Filtro

- auth | grupo | Can change group
- auth | grupo | Can delete group
- auth | grupo | Can view group
- auth | permiso | Can add permission
- auth | permiso | Can change permission
- auth | permiso | Can delete permission
- auth | permiso | Can view permission
- contacto | contacto | Can add contacto
- contacto | contacto | Can change contacto
- contacto | contacto | Can delete contacto
- contacto | contacto | Can view contacto
- contenttypes | tipo de contenido | Can add content type
- contenttypes | tipo de contenido | Can change content type

Seleccionar todos/as

permisos seleccionados/as

Filtro

- comentario | Can add comentario
- comentario | Can change comentario
- comentario | Can delete comentario
- comentario | Can view comentario

Eliminar todos/as

Y, haremos una modificación en la view.py de usuario para que cada usuario nuevo que se registre, tendrá asignado por defecto el perfil de usuario "Registrado", ya que a los usuarios "Colaborador" nosotros como "SuperUsuario" deberemos darle el acceso a ser "Colaborador".

```
apps > usuario > views.py > ...
1 from .forms import RegistroUsuarioForm
2 from django.contrib.auth.views import LoginView, LogoutView
3 from django.views.generic import CreateView
4 from django.contrib import messages
5 from django.shortcuts import redirect
6 from django.urls import reverse
7 from django.contrib.auth.views import PasswordResetView
8 from django.contrib.auth.views import PasswordResetDoneView
9 from django.contrib.auth.models import Group
10
11 # Create your views here.
12
13 class RegistrarUsuario(CreateView):
14     template_name = 'registration/regarstrar.html'
15     form_class = RegistroUsuarioForm
16
17     def form_valid(self, form):
18         response = super().form_valid(form)
19         messages.success(self.request, 'Registro exitoso. Por favor, inicia sesión.')
20         group = Group.objects.get(name='Registrado')
21         self.object.groups.add(group)
22         return redirect('apps.usuario:regarstrar')
```

Probamos haciendo un registro de usuario desde nuestra web y luego vamos al admin de Django para verificar que está funcionando correctamente.

The screenshot shows the Django Admin interface. On the left is a sidebar menu with categories like 'AUTENTICACIÓN Y AUTORIZACIÓN', 'CONTACTO', 'POSTS', and 'USUARIO'. The 'USUARIO' section is expanded, showing 'Usuarios' with a '+ Agregar' button. The main content area is titled 'Modificar usuario' and shows details for a user named 'Registrado'. The 'Contraseña' field is filled with a long alphanumeric string. Below it, the 'Último ingreso' section has fields for 'Fecha' and 'Hora'. A checkbox for 'Es superusuario' is unchecked. The 'Grupos' section shows a dropdown menu with 'Colaborador' and 'Registrado' selected. A red box highlights this dropdown menu. Below the dropdown, there is a note about permissions and a hint to use the Command key for multiple selection.

En este caso hicimos un registro de un nuevo usuario desde la web llamado "Registrado" y luego fuimos al admin de Django y nos dirigimos al usuario que se creó donde podemos ver que efectivamente, en la casilla de grupos, está marcado como "Registrado".

The screenshot shows a web page with a dark blue header containing the word 'OPINIO' and links for 'Inicio' and 'Acerca de nosotros'. Below the header is a section titled 'Comentarios'. It displays two comments: one from 'root' dated '2023' with the text 'Este es mi primer comentario', and another from 'Registrado' dated '2023' with the text 'Este es mi primer comentario como usuario Registrado!'. At the bottom, there is a section titled 'Deja tu comentario' with a text input field and a 'Comentar' button.

Y si nos dirigimos al post que tenemos verificamos que ya podemos realizar un comentario como usuario registrado:

Lo siguiente es crear el menú y accesos para cada usuario. Para ello deberemos armar templates donde el usuario Colaborador podrá crear "posts", modificarlos o eliminarlos, además podrá ver la lista de usuarios que hay registrados para eliminarlos si es necesario y, podrá realizar comentarios sobre los posts, modificar o eliminarlos tanto a los propios, y, los usuarios registrados podrán realizar comentarios en los posts (como lo acabamos de ver) además de modificar o eliminar sus propios comentarios.

Primero vamos a registrar un usuario "colaborador" y darle el acceso como "Colaborador"



Guardamos los cambios y comenzamos con las views para este usuario.

Aclaremos nuevamente, esta es solo una forma de hacerlo, es solo un ejemplo para poder guiare pero tu podrás adaptar a tu código como lo deses.

Aclarado esto, lo que haremos para que este usuario colaborador es darle un botón de acceso para la creación de "posts" desde la sección de "posts", donde cuando este usuario abra los posts, le aparezca un botón para crear posts, sin embargo, para el usuario Registrado no va a pasar lo mismo, este solo tendrá acceso a realizar comentarios.



Quedando algo similar a esto.

Lo primero será crear un formulario para realizar esta acción.

Este formulario, en este caso lo hacemos desde la app "post" pero tranquilamente se puede hacer desde la app "usuario".

Para este formulario, además de la creación de post, deberemos crear otro para agregar categorías:

```

blog > apps > posts > forms.py > ...
1  from django import forms
2  from .models import Comentario, Post, Categoria
3
4
5  class ComentarioForm(forms.ModelForm):
6      class Meta:
7          model = Comentario
8          fields = ['texto']
9
10 class CrearPostForm(forms.ModelForm):
11     class Meta:
12         model = Post
13         fields = '__all__'
14
15 class NuevaCategoriaForm(forms.ModelForm):
16     class Meta:
17         model = Categoria
18         fields = '__all__'
19
    
```

Como vamos a usar todos los campos de estos modelos, usamos la cláusula '___all__' (recuerda que siempre debemos importar lo que vamos a usar, en este caso los modelos Post y Categoría).

Un vez realizado esto, debemos crear la vista para que podamos usar estos formularios y para ello necesitaremos importar el modelo Categoría y los formularios CrearPostForm, NuevaCategoríaForm. Además, utilizaremos la función reverse_lazy, por lo que también debemos importarla.

Para la creación de los posts y categorías vamos a heredar de la vista CreateView.

```
apps > posts > views.py > ...
38 |         context = self.get_context_data(**kwargs)
39 |         context['form'] = form
40 |         return self.render_to_response(context)
41 |
42 | class PostCreateView(CreateView):
43 |     model = Post
44 |     form_class = CrearPostForm
45 |     template_name = 'posts/crear_post.html'
46 |     success_url = reverse_lazy('apps.posts:posts')
47 |
48 | class CategoriaCreateView(CreateView):
49 |     model = Categoria
50 |     form_class = NuevaCategoríaForm
51 |     template_name = 'posts/crear_categoria.html'
52 |
53 |     def get_success_url(self):
54 |         next_url = self.request.GET.get('next')
55 |         if next_url:
56 |             return next_url
57 |         else:
58 |             return reverse_lazy('apps.posts:post_create')
59 |
60 | class ComentarioCreateView(LoginRequiredMixin, CreateView):
```

Para las categorías incluimos la función get_success_url, para que nos vuelva a redigir a la página en la que estábamos una vez creada una nueva categoría.

Además, puedes ver que tanto para Post como para Categoría declaramos los html que vamos a usar por lo que vamos a crearlos dentro de la carpeta templates/posts, sin embargo, para seguir el mismo orden, crearemos las urls primero.

```
apps > posts > urls.py > ...
1 | from django.urls import path
2 | from .views import PostListView, PostDetailView, PostCreateView, CategoriaCreateView
3 |
4 | app_name = 'apps.posts'
5 |
6 | urlpatterns = [
7 |     path('posts/', PostListView.as_view(), name='posts'),
8 |     path('posts/<int:id>/', PostDetailView.as_view(), name='post_individual'),
9 |     path('post/', PostCreateView.as_view(), name='crear_post'),
10 |    path('post/categoria', CategoriaCreateView.as_view(), name='crear_categoria'),
11 | ]
```

Ahora crearemos los templates de crear_post.html y crear_categoria.html:

```
templates > posts > crear_post.html > ...
1 | {% extends 'base.html' %}
2 |
3 | {% block contenido %}
4 |
5 | <center>
6 |     <div class="container-fluid" style="margin: 200px;">
7 |         <div class="container-fluid" style="margin-top: 300px;">
8 |             <a id="boton_post" href="{% url 'apps.posts:crear_categoria' %}">Crear nueva categoría</a>
9 |         </div>
10 |
11 |         <h1>Crear Post</h1>
12 |         <form method="POST" enctype="multipart/form-data">
13 |             {% csrf_token %}
14 |             {{ form.as_p }}
15 |             <input type="submit" value="Guardar">
16 |         </form>
17 |
18 |     </div>
19 | </center>
20 | {% endblock %}
```

```
templates > posts > crear_categoria.html > ...
1 | {% extends 'base.html' %}
2 |
3 | {% block contenido %}
4 |
5 | <center>
6 |     <div class="container-fluid" style="margin: 200px;">
7 |         <h1>Agregar nueva categoría</h1>
8 |         <form method="post">
9 |             {% csrf_token %}
10 |             {{ form.as_p }}
11 |             <input type="submit" value="Agregar">
12 |         </form>
13 |     </div>
14 | </center>
15 | {% endblock %}
```

Ahora incluiremos el botón para acceder desde post.html:

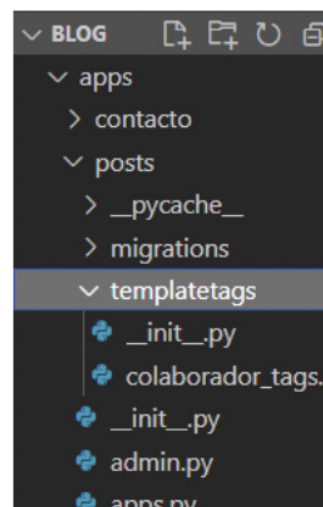
```
templates > posts > posts.html > ...
1 | {% extends 'base.html' %}
2 | {% load static %}
3 |
4 | {% block contenido %}
5 |
6 | <div class="container-fluid" style="margin: 200px;">
7 |
8 | <div class="container-fluid" style="margin-top: 300px;">
9 |     <a id="boton_post" href="{% url 'apps.posts:crear_post' %}">Crear nuevo post</a>
10 | </div>
11 |
12 | {% for i in posts %}
13 |     <br>
```

Ejecutamos el servidor y probamos.



Si lo has probado, tendrás un resultado similar al nuestro, sin embargo, también te habrás dado cuenta que el acceso está abierto para todos, y para que solo sea para usuarios colaboradores o super usuario, tendremos que crear tags y limitar el acceso (recuerda que esto es solo una forma de hacerlo para orientarte, puedes elegir otra forma de hacerlo investigando... te volvemos a recalcar una vez más que, en la programación es necesario leer, estudiar e investigar por cuenta propia para nutrirse de más conocimiento y lograr realizar una misma cosa, pero de diversas formas, más simples o más complejas, según lo que se requiera).

Para realizar estos tags, vamos a crear una nueva carpeta llamada "templatetags" dentro de nuestra aplicación. En este caso en la app posts.



Dentro de esta carpeta pondremos un archivo "__init__.py" vacío (es decir, adentro de él no irá nada) para que Django lea si o si esta carpeta y otro archivo "colaborador_tags.py" donde, dentro de él pondremos el filtro necesario para que el botón que acabamos de crear, solo aparezca cuando sea un usuario "Colaborador".

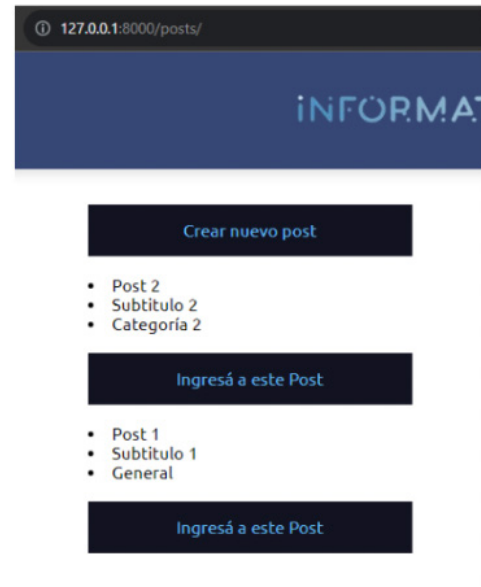
```
apps > posts > templatetags > colaborador_tags.py > ...
1 from django import template
2
3 register = template.Library()
4
5 @register.filter(name='has_group')
6 def has_group(user, group_name):
7     return user.groups.filter(name=group_name).exists()
```

Volvemos a la plantilla posts.html y cargamos el tags. Luego, definimos un condicional para mostrar el botón solo si el usuario es "Colaborador" o "super usuario" (sin embargo, super usuario puede estar como no, solo lo pusimos para mostrarte que se pueden poner distintos usuarios a filtrar con el condicional usando "or", e incluso, si necesitas cumplir más condiciones, puedes usar "and" tal cual lo hacemos en Python).

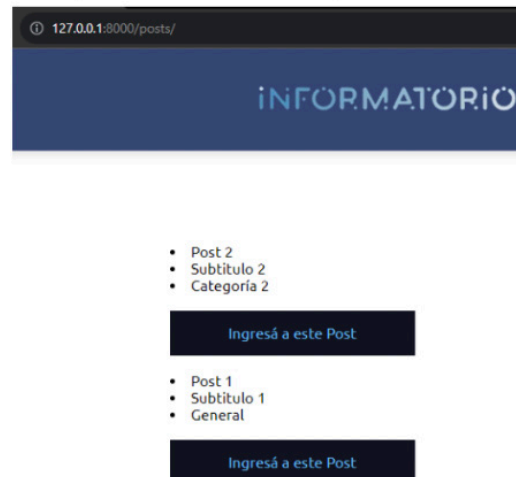
```
templates > posts > posts.html > ...
1 {% extends 'base.html' %}
2 {% load static %}
3 {% load colaborador_tags %}
4
5 {% block contenido %}
6
7 <div class="container-fluid" style="margin: 200px;">
8
9     {% if user.is_superuser or request.user|has_group:"Colaborador" %}
10         <div class="container-fluid" style="margin-top: 300px;">
11             <a id="boton_post" href="{% url 'apps.posts:crear_post' %}">Crear nuevo post</a>
12         </div>
13     {% endif %}
14
15     {% for i in posts %}
```

Ahora para probar, si no detuviste el servidor mientras hacías estas modificaciones, deténlo y vuelve a ejecutar, luego lo probaremos con los usuarios que tenemos registrados -- "Colaborador1", "Registrado" y "root".

root y Colaborador



Registrado y no registrado



Ahora haremos un listado de categorías donde el usuario Colaborador podrá eliminar las categorías que no quiera o modificarlas. Para ellos, y porque estamos en la carpeta templates/posts, vamos a crear dos plantillas nuevas "categoria_list.html" y "categoria_confirm_delete.html":

```
templates > posts > categoria_list.html > ...
1 {% extends 'base.html' %}
2
3 {% block contenido %}
4 <center>
5 <div class="container-fluid" style="margin: 200px;">
6 <h1>Categorías</h1>
7 <br>
8 <ul>
9     {% for categoria in categorias %}
10         <li>{{ categoria.nombre }}
11         <a style="color: black;"
12         href="{% url 'apps.posts:categoria_delete' pk=categoria.pk %}">Eliminar</a></li>
13     {% empty %}
14         <li>No hay categorías</li>
15     {% endfor %}
16 </ul>
17 </div>
18 </center>
19 {% endblock %}
```

```
templates > posts > categoria_confirm_delete.html > ...
1 {% extends 'base.html' %}
2
3 {% block contenido %}
4 <center>
5 <div class="container-fluid" style="margin: 300px;">
6 <h1>Eliminar Categoría</h1>
7 <p>¿Estás seguro de que quieres eliminar la categoría "{{ object.nombre }}"?</p>
8 <form method="post">
9     {% csrf_token %}
10     <input type="submit" value="Sí, eliminar">
11     <a style="color: black;" href="{% url 'apps.posts:categoria_list' %}">No, cancelar</a>
12 </form>
13 </div>
14 </center>
15 {% endblock %}
```

Donde en el template de la lista de categorías pondremos un botón de borrar las categorías en el cual la sintaxis genera una URL para la vista "categoria_delete" (que ahora crearemos) de nuestra app, pasando el valor de "categoria.pk" como argumento. Ahora iremos a crear las vistas, donde usaremos vistas que nos provee Django (vamos a importar la vista DeleteView, al lado de vista DetailView):

```
apps > posts > views.py > ...
1 from .models import Post, Comentario, Categoria
2 from .forms import ComentarioForm, CrearPostForm, NuevaCategoriaForm
3 from django.views.generic import ListView, DetailView, DeleteView
4 from django.contrib.auth.mixins import LoginRequiredMixin
5 from django.views.generic.edit import CreateView
6 from django.shortcuts import redirect
7 from django.urls import reverse_lazy
8
9
10 > class PostListView(ListView):...
14
15 > class PostDetailView(DetailView):...
41
42 > class PostCreateView(CreateView):...
47
48 > class CategoriaCreateView(CreateView):...
59
60 class CategoriaListView(ListView):
61     model = Categoria
62     template_name = 'posts/categoria_list.html'
63     context_object_name = 'categorias'
64
65 class CategoriaDeleteView(DeleteView):
66     model = Categoria
67     template_name = 'posts/categoria_confirm_delete.html'
68     success_url = reverse_lazy('apps.posts:categoria_list')
```

Notarás que prácticamente venimos usando código muy similar para estas nuevas vistas que estamos creando. Esto es gracias a que seguimos reutilizando las vistas genéricas que nos provee Django.

Una vez creadas nuestras vistas, creamos las urls:

```
apps > posts > urls.py > ...
1 from django.urls import path
2 from .views import *
3
4 app_name = 'apps.posts'
5
6 urlpatterns = [
7     path('posts/', PostListView.as_view(), name='posts'),
8     path('posts/<int:id>/', PostDetailView.as_view(), name='post_individual'),
9     path('post/', PostCreateView.as_view(), name='crear_post'),
10    path('post/categoria', CategoriaCreateView.as_view(), name='crear_categoria'),
11    path('categoria/', CategoriaListView.as_view(), name='categoria_list'),
12    path('categoria/<int:pk>/delete/', CategoriaDeleteView.as_view(), name='categoria_delete'),
13 ]
```

Cambiamos por el * para traer todo porque la lista de vistas se hacía muy larga

Volvemos a nuestra plantilla de "crear_categoria" para insertar un acceso a la lista de categorías:

```
templates > posts > crear_categoria.html > ...
1 {% extends 'base.html' %}
2
3 {% block contenido %}
4
5 <center>
6 <div class="container-fluid" style="margin: 200px;">
7     <h1>Agregar nueva categoría</h1>
8     <form method="post">
9         {% csrf_token %}
10         {{ form.as_p }}
11         <input type="submit" value="Agregar">
12     </form>
13     <div class="container-fluid" style="margin-top: 300px;">
14         <a id="boton_post"
15         href="{% url 'apps.posts:categoria_list' %}">Listar todas las categorías</a>
16     </div>
17 </div>
18 </center>
19 {% endblock %}
```

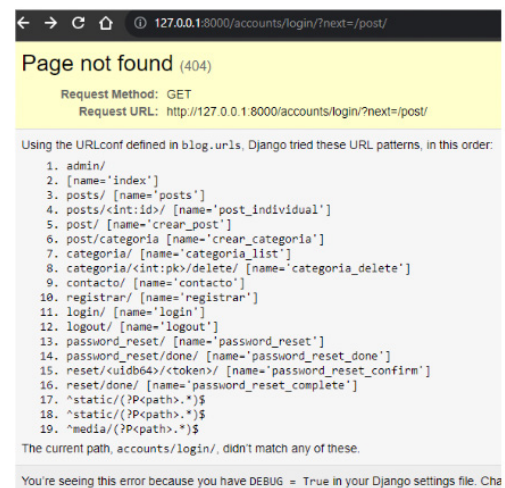
Lo probamos:

Ahora, si llegaras a desloguearte e ingresas la en la dirección http, por ejemplo "http://127.0.0.1:8000/post/", verás que se puede acceder sin estar logueado, por lo que esto aún sigue liberado para el acceso de cualquier usuario, así que vamos a cerrar para solo los usuarios colaboradores y super usuario.

Vamos a usar "LoginRequiredMixin" como lo hicimos con comentarios, pero también puedes usar decoradores.

```
apps > posts > views.py > ...
1 from .models import Post, Comentario, Categoria
2 from .forms import ComentarioForm, CrearPostForm, NuevaCategoriaForm
3 from django.views.generic import ListView, DetailView, DeleteView
4 from django.contrib.auth.mixins import LoginRequiredMixin
5 from django.views.generic.edit import CreateView
6 from django.shortcuts import redirect
7 from django.urls import reverse_lazy
8
9
10 > class PostListView(ListView): ...
14
15 > class PostDetailView(DetailView): ...
41
42 > class PostCreateView(LoginRequiredMixin, CreateView): ...
47
48 > class CategoriaCreateView(LoginRequiredMixin, CreateView): ...
59
60 > class CategoriaListView(ListView): ...
64
65 > class CategoriaDeleteView(LoginRequiredMixin, DeleteView): ...
69
70
71 > class ComentarioCreateView(LoginRequiredMixin, CreateView): ...
79
```

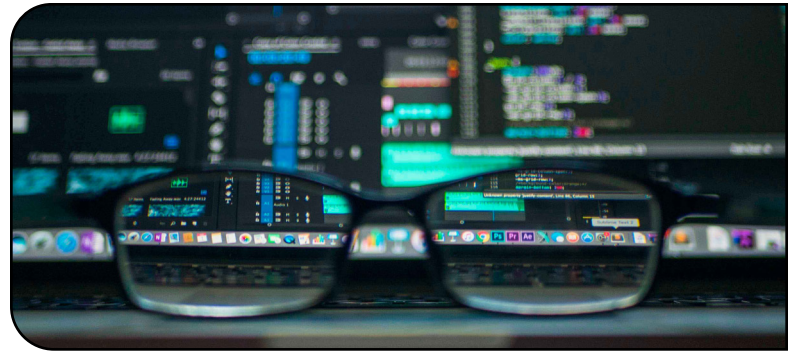
Lo único que hacemos es agregar la herencia de "LoginRequiredMixin" a las vistas a restringir, por lo que si ahora lo pruebas nuevamente, de poner la dirección /post (luego de guardar los cambios), no tendrás acceso.



De todas formas, haremos más adelante, otras modificaciones para que los usuarios Registrados no puedan acceder tampoco si llegan a poner la dirección en el navegador (pruébalos y verás que si estás logueado como usuario Registrado, podrás de igual manera, crear un post). También a modo de prueba, podríamos tratar el error 404 de "Page not found", de esta manera, seguiremos organizando nuestro proyecto para lo que es el deploy final al servidor. Así que con una pequeña pausa, a lo que venimos haciendo, vamos a hacer una página personalizada para "Page not found". Para esto vamos a realizar las siguientes configuraciones, dirigiendonos primero que nada a settings.py:

> Personalizando 404

Page not found - 404 page not found - 404



```
blog > blog > configuraciones > settings.py > ...
25 # SECURITY WARNING: don't run with debug turned on in production!
26 DEBUG = True
27
28 ALLOWED_HOSTS = []
```

Llevamos estas configuraciones a local.py

```
blog > configuraciones > local.py > ...
1 from .settings import *
2
3 DEBUG = False
4
5 ALLOWED_HOSTS = ['localhost', '127.0.0.1']
6
7 DATABASES = {
8     'default': {
```

Donde para tratar la página 404, de manera local, y, a modo de prueba ponemos el DEBUG en False y en ALLOWED_HOST la configuración del servidor con el que estamos trabajando.

Estas configuraciones también las pondremos en prod.py ya que nos servirá para el deploy del proyecto al final.

```
blog > configuraciones > prod.py > ...
1 from .settings import *
2
3 DEBUG = False
4
5 ALLOWED_HOSTS = []
6
```

En prod.py si dejaremos completamente en False al DEBUG, ya que si lo dejamos en True como lo tenemos en este momento en modo local, es un gran riesgo para la seguridad de nuestro sitio. Es como dejar la puerta abierta de nuestra casa en una zona con problemas de seguridad.

En local.py por el momento dejaremos en False a DEBUG, solo para mostrar la personalización de la página 404, pero, luego la volveremos a True para seguir trabajando correctamente.

Crearemos una vista personalizada para el template 404. Para ellos vamos a dirigirnos a nuestra views principal y crearemos la siguiente vista:

```
blog > views.py > ...
1 from django.shortcuts import render
2 from django.http import HttpResponseRedirect
3
4 def index(request):
5     return render(request, 'index.html')
6
7 def pagina_404(request, exception):
8     return HttpResponseRedirect('<h1>Página no encontrada</h1>')
9
```

Luego generaremos la url, de nuestra url principal, para asignar la función que creamos en la view.

```
blog > urls.py > ...
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16     """
17     from django.contrib import admin
18     from django.urls import path, include
19     from .views import index, pagina_404
20     from django.conf import settings
21     from django.conf.urls.static import static
22     from django.contrib.staticfiles.urls import staticfiles_urlpatterns
23
24     handler404 = pagina_404
25
26     urlpatterns = [
27         path('admin/', admin.site.urls),
28         path('', index, name='index'),
29         path('', include('apps.posts.urls')),
30         path('', include('apps.contacto.urls')),
31         path('', include('apps.usuario.urls')),
32     ]+static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
33     urlpatterns += staticfiles_urlpatterns()
34     urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Asignamos la función de vista para el template 404 al atributo handler404. Guardamos todos los cambios y actualizamos la página en la que estábamos.



Esta es una simple muestra de cómo puedes tratar los errores que puede tener la página y también es una forma de no dejar "abierta" alguna puerta a ataques que pueda recibir nuestra página.

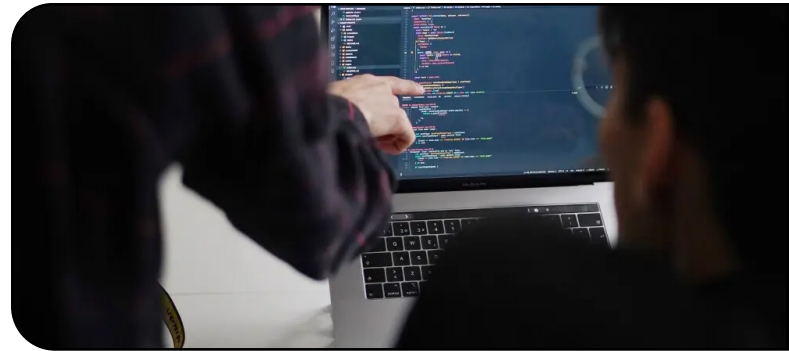
Para seguir trabajando bien y que Django no brinde los errores de manera correcta pudiendo visualizar que está ocurriendo si tenemos errores, vamos a activar nuevamente el DEBUG en True.

```
blog > configuraciones > local.py > ...
1     from .settings import *
2
3     DEBUG = True
4
```

Y proseguiremos con lo próximo que sería la modificación de posts para nuestro usuario Colaborador.

> Permisos de usuario

Usuario colaborador - usuario registrado



Para realizar la modificación de algún post que se requiera siendo usuario Colaborador, vamos a utilizar otra vista que nos brinda Django. `UpdateView`. Nos dirigimos a nuestra `view.py` de la app `posts` e importamos la vista `UpdateView`. Luego creamos la nueva vista para la modificación de los posts:

```
apps > posts > urls.py ...
1 from django.urls import path
2 from .views import *
3
4 app_name = 'apps.posts'
5
6 urlpatterns = [
7     path('posts/', PostListView.as_view(), name='posts'),
8     path('posts/<int:id>/', PostDetailView.as_view(), name='post_individual'),
9     path('post/', PostCreateView.as_view(), name='crear_post'),
10    path('post/categoria', CategoriaCreateView.as_view(), name='crear_categoria'),
11    path('categoria/', CategoriaListView.as_view(), name='categoria_list'),
12    path('categoria/<int:pk>/delete/', CategoriaDeleteView.as_view(), name='categoria_delete'),
13    path('post/<int:pk>/modificar/', PostUpdateView.as_view(), name='post_update'),
14 ]
```

Y ahora creamos el template.

```
templates > posts > modificar_post.html ...
1 {% extends 'base.html' %}
2
3 {% block contenido %}
4
5 <center>
6 <div class="container-fluid" style="margin: 30px;">
7     <a id="boton_post"
8       href="{% url 'apps.posts:crear_categoria' %}?next={% url 'apps.posts:post_update' pk=object.id %}">
9       Crear nueva categoria</a>
10    <br>
11    <h1>Modificar post</h1>
12    <form method="post" enctype="multipart/form-data">
13        {% csrf_token %}
14        {{ form.as_p }}
15        <input type="submit" value="Guardar cambios">
16    </form>
17 </div>
18 </center>
19
20 {% endblock %}
```

```
apps > posts > views.py ...
1 from .forms import ComentarioForm, CrearPostForm, NuevaCategoriaForm
2 from django.views.generic import ListView, DetailView, DeleteView, UpdateView
3 from django.contrib.auth.mixins import LoginRequiredMixin
4 from django.views.generic.edit import CreateView
5 from django.shortcuts import redirect
6 from django.urls import reverse_lazy
7
8
9
10 class PostListView(ListView): ...
11
12
13
14
15 > class PostDetailView(DetailView): ...
16
17
18
19
20 > class PostCreateView(LoginRequiredMixin, CreateView): ...
21
22
23
24
25 > class CategoriaCreateView(LoginRequiredMixin, CreateView): ...
26
27
28
29
30 > class CategoriaListView(ListView): ...
31
32
33
34
35 > class CategoriaDeleteView(LoginRequiredMixin, DeleteView): ...
36
37
38
39
40 class PostUpdateView(LoginRequiredMixin, UpdateView):
41     model = Post
42     form_class = CrearPostForm
43     template_name = 'posts/modificar_post.html'
44     success_url = reverse_lazy('apps.posts:posts')
45
46
47 > class ComentarioCreateView(LoginRequiredMixin, CreateView): ...
48
```

Donde al igual que las demás vistas, vamos a hacer una herencia de `LoginRequiredMixin` y de la nueva vista que utilizaremos `UpdateView`.

A su vez, usaremos una plantilla que llamamos "modificar_posts.html". Crearemos la url y luego el template.

En nuestro botón vamos a utilizar el "href" para ir a la url de crear categoría pero también agregaremos un parámetro `next` a la url utilizando la sintaxis `?next={% url 'apps.posts:post_update' pk=object.id %}` para que después de crear la categoría, se redirija al mismo post que estamos modificando.

Lo siguiente del código es igual que la creación de un post.

Este acceso lo agregaremos en "post_individual.html", el cual, se habilitará si el usuario es Colaborador (también puedes agregar que el super usuario tenga el acceso, como te habíamos mostrado en la creación de post).


```

templates > posts > post_individual.html > center > div.container-fluid
1  {% extends 'base.html' %}
2  {% load static %}
3  {% load colaborador_tags %}
4
5  {% block contenido %}
6  <center>
7  <div class="container-fluid" style="margin: 200px;">
8
9      <li>{{ posts.titulo }}</li>
10     <li>{{ posts.subtitulo }}</li>
11     <li>{{ posts.categoria }}</li>
12     <br>
13     <li>{{ posts.texto }}</li>
14     
15
16     {% if user.is_superuser or request.user.has_group:"Colaborador" %}
17     <div class="container-fluid" style="margin-top: 300px;">
18         <a id="boton_post" href="{% url 'apps.posts.post_update' pk=posts.id %}">Modificar Post</a>
19     </div>
20     {% endif %}
21

```

Puedes probar y verás que ya está funcionando la modificación de un post si el usuario es Colaborador o super usuario.

Ahora debemos integrar un botón para eliminar el post, para ir finalizando con los templates para los posts con acceso Colaborador.

```

10     <li>{{ posts.subtitulo }}</li>
11     <li>{{ posts.categoria }}</li>
12     <br>
13     <li>{{ posts.texto }}</li>
14     {% if posts.imagen %}
15     
16     {% else %}
17     <p>No hay imagen disponible</p>
18     {% endif %}
19     {% if user.is_superuser or request.user.has_group:"Colaborador" %}
20     <div class="container-fluid" style="margin-top: 300px;">
21         <a id="boton_post" href="{% url 'apps.posts.post_update' pk=posts.id %}">Modificar Post</a>
22     </div>
23     <div class="container-fluid" style="margin-top: 300px;">
24         <a id="boton_post" href="{% url 'apps.posts.post_delete' pk=posts.pk %}">Eliminar Post</a>
25     </div>
26     {% endif %}
27 </div>

```

Por lo que, ya estamos aquí, vamos a insertarlo y realizar una modificación en las imágenes que no realizamos anteriormente. Esto es para que al igual que hicimos con los posts hace varias páginas, de que, si no hay posts, muestre alguna referencia, ya que puede ocurrir también que si no hay imagen, muestre alguna referencia.

Creamos la view y luego la url.

```

14
15 > class PostDetailView(DetailView):...
41
42 > class PostCreateView(LoginRequiredMixin, CreateView):...
47
48 > class CategoriaCreateView(LoginRequiredMixin, CreateView):...
59
60 > class CategoriListView(ListView):...
64
65 > class CategoriaDeleteView(LoginRequiredMixin, DeleteView):...
69
70 > class PostUpdateView(LoginRequiredMixin, UpdateView):...
75
76 class PostDeleteView(DeleteView):
77     model = Post
78     template_name = 'posts/eliminar_post.html'
79     success_url = reverse_lazy('apps.posts:posts')
80

```

```

apps > posts > urls.py > ...
1  from django.urls import path
2  from .views import *
3
4  app_name = 'apps.posts'
5
6  urlpatterns = [
7      path('posts/', PostListView.as_view(), name='posts'),
8      path('posts/<int:id>/', PostDetailView.as_view(), name='post_individual'),
9      path('posts/', PostCreateView.as_view(), name='crear_post'),
10     path('post/categoria/', CategoriaCreateView.as_view(), name='crear_categoria'),
11     path('categoria/', CategoriListView.as_view(), name='categoria_list'),
12     path('categoria/<int:pk>/delete/', CategoriaDeleteView.as_view(), name='categoria_delete'),
13     path('post/<int:pk>/modificar/', PostUpdateView.as_view(), name='post_update'),
14     path('post/<int:pk>/eliminar/', PostDeleteView.as_view(), name='post_delete'),
15 ]

```



Lo probamos y funciona correctamente. Si te llegara a salir algún error con respecto a la imagen, solo tienes que verificar que la imagen por defecto esté en el lugar correcto mediante el modelo de "post" el cual configuramos en la página 26. Si quieres cambiar de lugar en donde se guarda la imagen por defecto, tienes que cambiarlo desde ahí y hacer las migraciones para que surgan efectos los cambios.

Si has probado introducir la ruta de creación de un post u otras rutas estando como usuario Registrado, verás que aún se puede acceder. Por eso lo que nos queda es realizar las restricciones para que solo los usuarios Colaboradores puedan tener acceso y no los usuarios Registrados. Esto lo tienes que realizar en los templates que el usuario Registrado no puede tener acceso. Por ejemplo:

```

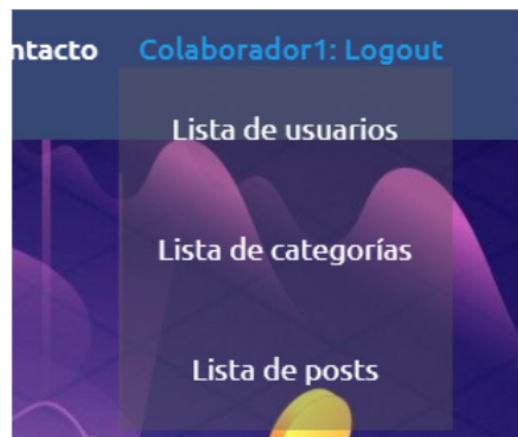
templates > posts > crear_post.html > ...
1  {% extends 'base.html' %}
2  {% load colaborador_tags %}
3
4  {% block contenido %}
5
6  <center>
7  {% if user.is_superuser or request.user.has_group:"Colaborador" %}
8  <div class="container-fluid" style="margin: 200px;">
9  <div class="container-fluid" style="margin-top: 300px;">
10     <a id="boton_post" href="{% url 'apps.posts:crear_categoria' %}">Crear nueva categoría</a>
11 </div>
12
13     <h1>Crear Post</h1>
14     <form method="POST" enctype="multipart/form-data">
15         {{ csrf_token }}
16         {{ form.as_p }}
17         <input type="submit" value="Guardar">
18     </form>
19 </div>
20 {% else %}
21 <div class="container-fluid" style="margin: 300px;">
22     <h1>Solo usuarios con permisos pueden acceder a esta página</h1>
23 </div>
24 {% endif %}
25 </center>
26 {% endblock %}

```



Ahora crearemos una lista de usuarios para el Colaborador pueda eliminar usuarios Registrados si así lo desea. Esta lista la vamos a incluir como un menú desplegable para enseñarte otra forma más que puedes usar para hacer todos tu templates personalizados para los usuarios con permisos especiales. Vamos a dirigirnos al template base.html donde vamos a agregar el usuario que está activo según el logueo.

```
templates > base.html > html
1  {% load static %}
2  {% load colaborador_tags %}
36  <ul class="nav-links">
37    <li><a href="{% url 'index' %}">Inicio</a></li>
38    <li><a href="#">Acerca de nosotros</a></li>
39    <li><a href="{% url 'apps.posts.posts' %}">Posts</a></li>
40    <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
41    <div class="dropdown">
42      {% if user.is_authenticated %}
43        <li><a href="{% url 'apps.usuario.logout' %}">{{ user.username }}: Logout</a></li>
44        <div class="dropdown-content">
45          <table>
46            <tr>
47              {% if user.is_superuser or request.user.has_group("Colaborador" %}
48                <td><a href="{% url 'apps.usuario.usuario_list' %}">Lista de usuarios</a></td>
49              {% endif %}
50            </tr>
51            <tr>
52              <td><a href="#">Lista de categorías</a></td>
53            </tr>
54            <tr>
55              <td><a href="#">Lista de posts</a></td>
56            </tr>
57          </table>
58        </div>
59      {% else %}
60        <li><a href="{% url 'apps.usuario.login' %}">Login</a></li>
61      {% endif %}
62    </div>
```



```
templates > base.html > html > body > header > nav > div.nav-content > ul.nav-links > li > a
33  <li><a href="{% url 'index' %}">Inicio</a></li>
34  <li><a href="#">Acerca de nosotros</a></li>
35  <li><a href="{% url 'apps.posts.posts' %}">Posts</a></li>
36  <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
37  {% if user.is_authenticated %}
38    <li><a href="{% url 'apps.usuario.logout' %}">{{ user.username }}: Logout</a></li>
39  {% else %}
40    <li><a href="{% url 'apps.usuario.login' %}">Login</a></li>
41  {% endif %}
42  </ul>
43  </div>
44  </nav>
```

Donde vamos a agregar una pequeña sintaxis para que se pueda ver el usuario que está Logueado en este momento `{{ user.username }}`



Si lo probamos, ya se puede ver que el nombre del usuario que está logueado sale en el Nav-bar. Ahora crearemos un menú desplegable:

Usamos un poco de css para hacer un menú dropdown (en base.html) al acercar el mouse. Luego vamos a crear la vista para hacer la lista de usuarios. Esto básicamente será como la vista y url que hicimos para las categorías. Te mostramos la view dentro de la app "usuario":

```
apps > usuario > views.py > ...
50
51  class UsuarioListView(LoginRequiredMixin, ListView):
52      model = Usuario
53      template_name = 'usuario/usuario_list.html'
54      context_object_name = 'usuarios'
55
56      def get_queryset(self):
57          queryset = super().get_queryset()
58          queryset = queryset.exclude(is_superuser=True)
59          return queryset
60
61  class UsuarioDeleteView(LoginRequiredMixin, DeleteView):
62      model = Usuario
63      template_name = 'usuario/eliminar_usuario.html'
64      success_url = reverse_lazy('apps.usuario:usuario_list')
```

En la vista de “UsuarioListView” usamos un “queryset” para no mostrar al super usuario en la lista así, ante cualquier cosa, un usuario Colaborador no podrá eliminarnos. Si lo probamos, con estas vistas ya podemos eliminar los usuarios, sin embargo, también vamos a necesitar eliminar comentarios o posts en caso que se requiera. Por lo que usando una función sobre la vista del borrado de usuario, haremos las consultas pertinentes para eliminar o no los posts o comentarios del usuario.

```
apps > usuario > urls.py > ...
1  from django.urls import path
2  from . import views
3  from .views import *
4  from django.contrib.auth import views as auth_views
5
6  app_name = 'apps.usuario'
7
8  urlpatterns = [
9      path('registrar/', views.RegistrarUsuario.as_view(), name='registrar'),
10     path('login/', LoginUsuario.as_view(), name='login'),
11     path('logout/', views.LogoutUsuario.as_view(), name='logout'),
12     path('password_reset/', MyPasswordResetView.as_view(), name='password_reset'),
13     path('password_reset/done/', MyPasswordResetDoneView.as_view(), name='password_reset_done'),
14     path('reset/<uidb64>/<token>/', auth_views.PasswordResetConfirmView.as_view(), name='password_reset_confirm'),
15     path('reset/done/', auth_views.PasswordResetCompleteView.as_view(), name='password_reset_complete'),
16     path('usuarios/', UsuarioListView.as_view(), name='usuario_list'),
17     path('usuario/<int:pk>/eliminar/', UsuarioDeleteView.as_view(), name='usuario_delete'),
18 ]
```



Para esto vamos a actualizar la vista de eliminación con la función para borrar posts o comentarios, pero también vamos a dar un contexto para que el mensaje de eliminar posts, aparezca solo para usuarios Colaborador que son los que tienen permisos de realizar posts.

```
apps > usuario > views.py > ...
61
62 class UsuarioDeleteView(LoginRequiredMixin, DeleteView):
63     model = Usuario
64     template_name = 'usuario/eliminar_usuario.html'
65     success_url = reverse_lazy('apps.usuario:usuario_list')
66
67     def get_context_data(self, **kwargs):
68         context = super().get_context_data(**kwargs)
69         colaborador_group = Group.objects.get(name='Colaborador')
70         es_colaborador = colaborador_group in self.object.groups.all()
71         context['es_colaborador'] = es_colaborador
72         return context
73
74     def post(self, request, *args, **kwargs):
75         eliminar_comentarios = request.POST.get('eliminar_comentarios', False)
76         eliminar_posts = request.POST.get('eliminar_posts', False)
77         self.object = self.get_object()
78         if eliminar_comentarios:
79             Comentario.objects.filter(usuario=self.object).delete()
80
81         if eliminar_posts:
82             Post.objects.filter(autor=self.object).delete()
83         messages.success(request, f'Usuario {self.object.username} eliminado correctamente')
84         return self.delete(request, *args, **kwargs)
```

Y como puedes observar, declaramos un nuevo campo para nuestro modelo de Post, por lo que también vamos a actualizar el modelo así podremos tener referencia de quien creó un Post.

```
apps > posts > models.py > Post
15 class Post(models.Model):
16     titulo = models.CharField(max_length=50, null=False)
17     subtitulo = models.CharField(max_length=100, null=True, blank=True)
18     fecha = models.DateTimeField(auto_now_add=True)
19     texto = models.TextField(null=False)
20     activo = models.BooleanField(default=True)
21     categoria = models.ForeignKey(Categoria, on_delete=models.SET_NULL, null=True, default='Sin categoría')
22     imagen = models.ImageField(null=True, blank=True, upload_to='media', default='static/post_default.png')
23     publicado = models.DateTimeField(default=timezone.now)
24     autor = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, null=True)
25
26     class Meta:
27         ordering = ('publicado',)
```

Una vez actualizado el campo, recuerda que debes hacer las migraciones para que los cambios surgan efectos en la base de datos.

Por último, actualizamos nuestro template de eliminación y probamos:

```
templates > usuario > <> eliminar_usuario.html > ...
1  {% extends 'base.html' %}
2  {% load colaborador_tags %}
3
4  {% block contenido %}
5  <center>
6  {% if user.is_superuser or request.user|has_group:"Colaborador" %}
7  <div class="container-fluid" style="margin: 400px;">
8      <h1>Eliminar usuario</h1><br>
9      <p>¿Estás seguro de que quieres eliminar el usuario "{{ object.username }}"?</p><br>
10     <form method="post">
11         {% csrf_token %}
12         <label for="eliminar_comentarios">¿Desea eliminar también los comentarios del usuario?</label>
13         <input type="checkbox" name="eliminar_comentarios" id="eliminar_comentarios">
14         <br><br>
15         {% if es_colaborador %}
16             <label for="eliminar_posts">¿Desea eliminar también los posts del usuario?</label>
17             <input type="checkbox" name="eliminar_posts" id="eliminar_posts">
18             <br><br>
19         {% endif %}
20
21         <input type="submit" value="Sí, eliminar">
22         <a style="color: black;" href="{% url 'apps.usuario:usuario_list' %}">No, cancelar</a>
23     </form>
24 </div>
25 {% else %}
26     <div class="container-fluid" style="margin: 300px;">
27         <h1>Solo usuarios con permisos pueden acceder a esta página</h1>
28     </div>
29 {% endif %}
30 </center>
31 {% endblock %}
```

Inicio Acerca de nosotros Po

Eliminar usuario

¿Estás seguro de que quieres eliminar el usuario "Colaborador1"?

¿Desea eliminar también los comentarios del usuario? ☐

¿Desea eliminar también los posts del usuario? ☐

No, cancelar

Inicio Acerca de nosotros Po

Eliminar usuario

¿Estás seguro de que quieres eliminar el usuario "Registrado"?

¿Desea eliminar también los comentarios del usuario? ☐

No, cancelar

Ahora ya tenemos la eliminación de usuarios y con ellos también se pueden eliminar o no los posts para Colaborador y/o comentarios. Y si el usuario es Registrado, solo veremos la eliminación de comentarios para tildar si se quiere eliminar o no. Esto nos lleva a poder modificar un comentario o eliminarlo.

Para ello vamos a usar las vistas de modificación y eliminación de Categoría las cuales modificaremos para usar con los comentarios (importaremos reverse, al lado de reverse_lazy):

```
apps > posts > views.py > ...
80
81 > class ComentarioCreateView(LoginRequiredMixin, CreateView): ...
89
90 class ComentarioUpdateView(LoginRequiredMixin, UpdateView):
91     model = Comentario
92     form_class = ComentarioForm
93     template_name = 'comentario/comentario_form.html'
94
95     def get_success_url(self):
96         next_url = self.request.GET.get('next')
97         if next_url:
98             return next_url
99         else:
100             return reverse('apps.posts:post_individual', args=[self.object.posts.id])
101
102 class ComentarioDeleteView(LoginRequiredMixin, DeleteView):
103     model = Comentario
104     template_name = 'comentario/comentario_confirm_delete.html'
105
106     def get_success_url(self):
107         return reverse('apps.posts:post_individual', args=[self.object.posts.id])
108
```

Y para las urls:

```
apps > posts > urls.py > ...
1 from django.urls import path
2 from .views import *
3
4 app_name = 'apps.posts'
5
6 urlpatterns = [
7     path('posts/', PostListView.as_view(), name='posts'),
8     path('posts/<int:id>/', PostDetailView.as_view(), name='post_individual'),
9     path('post/', PostCreateView.as_view(), name='crear_post'),
10    path('post/categoria', CategoriaCreateView.as_view(), name='crear_categoria'),
11    path('categoria/', CategoriaListView.as_view(), name='categoria_list'),
12    path('categoria/<int:pk>/delete/', CategoriaDeleteView.as_view(), name='categoria_delete'),
13    path('post/<int:pk>/modificar/', PostUpdateView.as_view(), name='post_update'),
14    path('post/<int:pk>/eliminar/', PostDeleteView.as_view(), name='post_delete'),
15    path('comentario/<int:pk>/editar/', ComentarioUpdateView.as_view(), name='comentario_editar'),
16    path('comentario/<int:pk>/eliminar/', ComentarioDeleteView.as_view(), name='comentario_eliminar'),
17]
```

Como puedes ver, estas vistas y urls, son muy similares entre unas y otras ya que al utilizar las vistas genericas de Django, este se encarga de hacer casi todo el trabajo, lo único que nos queda a nosotros, es definir funciones para varias ciertos comportamientos referidos a lo que necesitamos hacer.

Te habrás dado cuenta que también comenzamos a mover los templates de comentarios a una carpeta llamada "comentario" para poder organizar mejor el código, ya que comenzamos a usar más plantillas para comentarios. Otra cosa que hicimos para enseñarte, es la de usar una incrustación de template dentro de otro template. Esto lo hicimos en la plantilla de post_individual.html donde teníamos los comentarios, para separar el código. Utilizamos la etiqueta "include" para llamar a un template desde otro template. Quedando así:

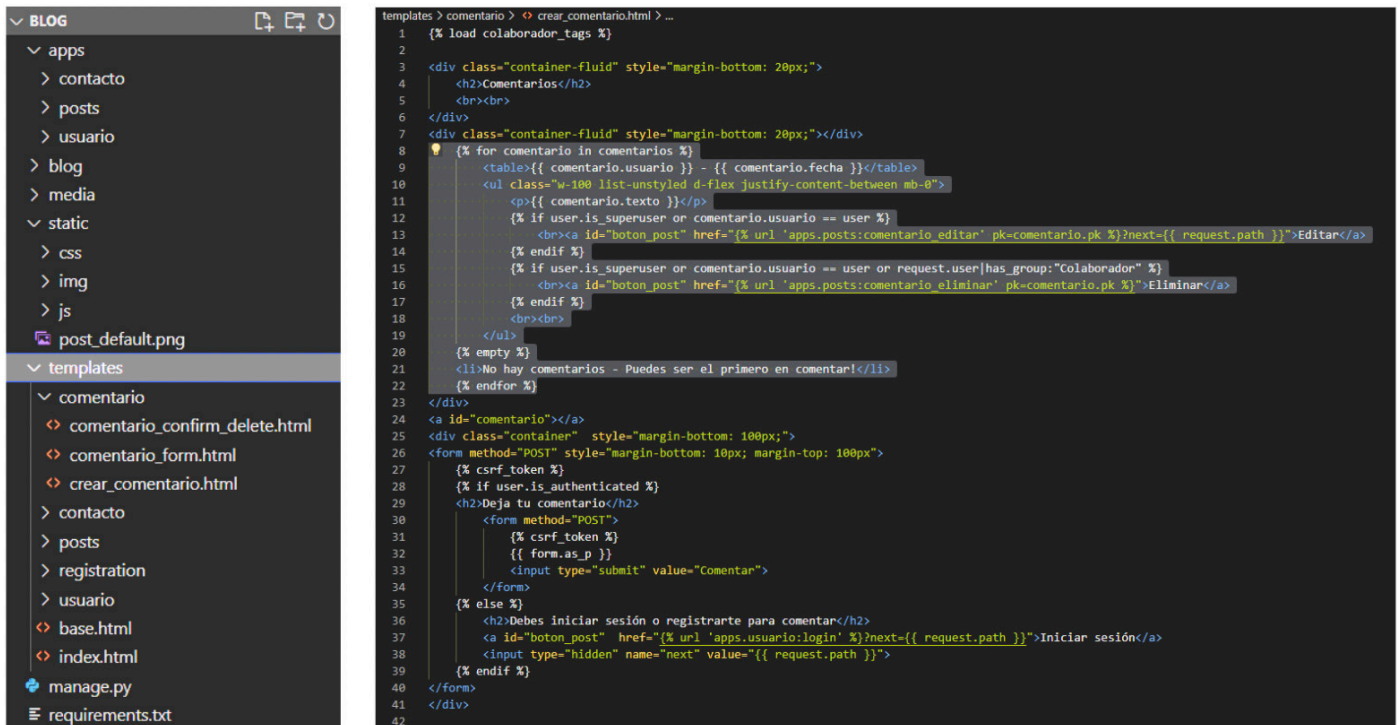
Mediante esta etiqueta, puedes separar el código para hacerlo aún más legible y fácil de trabajar, llamando desde una plantilla a otra plantilla.

Además verás que, para los comentarios, creamos este template "crear_comentario.html" en la carpeta comentario al mismo nivel de las demás carpetas dentro de la carpeta templates. Dentro de crear_comentario se encuentra el mismo código que había aquí, pero un poco modificado para cumplir con lo que requerimos.

```

templates > posts > post_individual.html > center
1  {% extends 'base.html' %}
2  {% load static %}
3  {% load colaborador_tags %}
4
5  {% block contenido %}
6  <center>
7  <div class="container-fluid" style="margin: 200px;">
8
9      <li>{{ posts.titulo }}</li>
10     <li>{{ posts.subtitulo }}</li>
11     <li>{{ posts.categoria }}</li>
12     <br>
13     <li>{{ posts.texto }}</li>
14     {% if posts.imagen %}
15     
16     {% else %}
17     <p>No hay imagen disponible</p>
18     {% endif %}
19     {% if user.is_superuser or request.user|has_group:"Colaborador" %}
20     <div class="container-fluid" style="margin-top: 300px;">
21         <a id="boton_post" href="{% url 'apps.posts:post_update' pk=posts.id %}">Modificar Post</a>
22     </div>
23     <div class="container-fluid" style="margin-top: 300px;">
24         <a id="boton_post" href="{% url 'apps.posts:post_delete' pk=posts.pk %}">Eliminar Post</a>
25     </div>
26     {% endif %}
27 </div>
28
29 <!-- Aquí seguirán estando los comentarios, pero
30     lo separamos en otro template y lo incluimos en este -->
31
32 {% include 'comentario/crear_comentario.html' %}
33
34 </center>
35 {% endblock %}
  
```

Te dejamos de muestra la estructura de como quedaron las carpetas. Ahora veremos el template de crear_comentario.html con sus modificaciones:



En esta primer parte lo que hacemos es agregar los botones de eliminar o modificar comentarios (el resto del código es el mismo que teníamos).

La salvedad que hacemos está relacionada para que el super usuario pueda realizar todo, tanto eliminar o modificar cualquier comentario (esto es opcional ya que el super usuario puede realizar todas las modificaciones que requiera desde el admin de Django).

Luego, para el usuario Colaborador designamos que pueda eliminar cualquier comentario, pero solo pueda modificar los suyos y no los de los usuarios Registrados, aunque con una simple modificación en el template puedes hacer que pueda editar también todos los comentarios, propios y de los Registrados.

Y, los usuarios Registrados podrán eliminar o modificar sus propios comentarios.

Cabe destacar que si hay más de un usuario Colaborador, podrá modificar, además de eliminar, cualquier comentario de otros usuarios Colaboradores, ya que en la sentencia solo estamos llamando al grupo de Colaborador, pero no al Colaborador específico, a diferencia de lo que hacemos con los usuarios Registrados. Esto lo puedes ir manejando en la medida que lo requieras, ya que desde aquí te damos las bases y diversas formas sencillas de hacerlo.

Otra cosa para destacar, es que aquí estamos dando accesos desde el propio template usando código Python, sin embargo, esto se puede hacer desde las views.

El otro template que usamos para realizar las modificaciones de los comentarios es el siguiente:

```
templates > comentario > comentario_form.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4  <center>
5      {% if user.is_superuser or comentario.usuario == user %}
6      <div class="container-fluid" style="margin: 300px;">
7          <h1>Editar comentario</h1>
8          <form method="POST" style="margin-bottom: 10px; margin-top: 100px">
9              {% csrf_token %}
10             {{ form.as_p }}
11             <input type="submit" value="Guardar cambios">
12          </form>
13      </div>
14      {% else %}
15          <div class="container-fluid" style="margin: 300px;">
16              <h1>Solo usuarios con permisos pueden acceder a esta página</h1>
17          </div>
18      {% endif %}
19  </center>
20  {% endblock %}
```

Y el template para eliminar comentarios:

```
templates > comentario > comentario_confirm_delete.html > ...
1  {% extends 'base.html' %}
2  {% load colaborador_tags %}
3
4  {% block contenido %}
5  <center>
6      {% if user.is_superuser or request.user|has_group:"Colaborador" or comentario.usuario == user %}
7      <div class="container-fluid" style="margin: 300px;">
8          <h1>Eliminar comentario</h1>
9          <p>¿Estás seguro de que deseas eliminar este comentario?</p>
10         <form method="POST">
11             {% csrf_token %}
12             <input type="submit" value="Sí, eliminar">
13             <a style="color: black;" href="{{ object.get_absolute_url }}">No, cancelar</a>
14         </form>
15     </div>
16     {% else %}
17         <div class="container-fluid" style="margin: 300px;">
18             <h1>Solo usuarios con permisos pueden acceder a esta página</h1>
19         </div>
20     {% endif %}
21 </center>
22 {% endblock %}
```

Hecho todo lo necesario para poder visualizar, te mostramos los resultados:



Aquí estamos logueados como Colaborador y puedes ver que el Colaborador puede eliminar los comentarios de los usuarios Registrados pero no modificarlos, sin embargo, puede modificar y eliminar los comentarios propios y de otros Colaboradores.

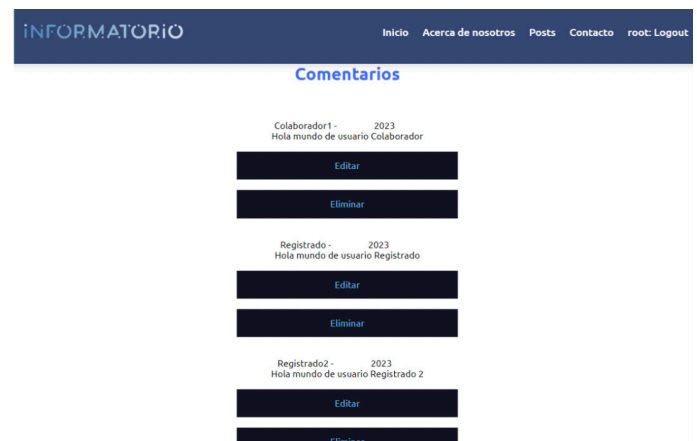


Luego hacemos un logueo como usuario Registrado.

Hecho todo lo necesario para poder visualizar, te mostramos los resultados:



Lo mismo para otro usuario Registrado, solo puede eliminar y editar su propio comentario.



Y el opcional para el super usuario que puede editar o eliminar cualquier comentario.

> Filtrado

Agregando algunos filtros más



Ahora vamos a enlazar los accesos que hicimos para el menú dropdown de base.html de manera tal que el usuario pueda filtrar los posts por categoría e ingresar a la lista de posts por esa categoría, para esto realizamos una pequeña modificación a base.html:

```
templates > base.html > html > body
27 <header>
28 <nav>
29 <div class="nav-content">
30 <div class="logo">
31 
32 </div>
33 <ul class="nav-links">
34 <li><a href="{% url 'index' %}">Inicio</a></li>
35 <li><a href="#">Acerca de nosotros</a></li>
36 <li><a href="{% url 'apps.posts:posts' %}">Posts</a></li>
37 <li><a href="{% url 'apps.contacto:contacto' %}">Contacto</a></li>
38 <div class="dropdown">
39 <div class="dropdown-content">
40 <div class="dropdown-content">
41 <div class="dropdown-content">
42 <div class="dropdown-content">
43 <div class="dropdown-content">
44 <div class="dropdown-content">
45 <div class="dropdown-content">
46 <div class="dropdown-content">
47 <div class="dropdown-content">
48 <div class="dropdown-content">
49 <div class="dropdown-content">
50 <div class="dropdown-content">
51 <div class="dropdown-content">
52 <div class="dropdown-content">
53 <div class="dropdown-content">
54 <div class="dropdown-content">
55 <div class="dropdown-content">
56 <div class="dropdown-content">
57 <div class="dropdown-content">
58 <div class="dropdown-content">
59 <div class="dropdown-content">
```

Con esto, todo usuario que navegue por la web podrá filtrar por categorías los posts. Si el usuario es colaborador, tendrá acceso a la lista de usuarios, y, todos los usuarios podrán loguearse, registrarse o desloguearse.

Además de esto tendremos que hacer una modificación en la vista de post para realizar el filtrado. Como siempre decimos, hay muchas formas de hacerlo. Nosotros crearemos una nueva vista para este filtrado.

Donde tomaremos como filtro las categorías mediante una queryset:

```
apps > posts > views.py > ...
59 class PostsPorCategoriaView(ListView):
60     model = Post
61     template_name = 'posts/posts_por_categoria.html'
62     context_object_name = 'posts'
63
64     def get_queryset(self):
65         return Post.objects.filter(categoria_id=self.kwargs['pk'])
66
```

Verás también que vamos a crear un nuevo template que será `posts_por_categoria.html`, pero primero configuraremos la url:

```
apps > posts > urls.py > ...
1 from django.urls import path
2 from .views import *
3
4 app_name = 'apps_posts'
5
6 urlpatterns = [
7     path('posts/', PostListView.as_view(), name='posts'),
8     path('posts/<int:id>/', PostDetailView.as_view(), name='post_individual'),
9     path('post/', PostCreateView.as_view(), name='crear_post'),
10    path('post/categoria', CategoriaCreateView.as_view(), name='crear_categoria'),
11    path('categoria/', CategoriaListView.as_view(), name='categoria_list'),
12    path('categoria/<int:pk>/delete/', CategoriaDeleteView.as_view(), name='categoria_delete'),
13    path('post/<int:pk>/modificar/', PostUpdateView.as_view(), name='post_update'),
14    path('post/<int:pk>/eliminar/', PostDeleteView.as_view(), name='post_delete'),
15    path('categoria/<int:pk>/posts/', PostsPorCategoriaView.as_view(), name='posts_por_categoria'),
16    path('comentario/<int:pk>/editar/', ComentarioUpdateView.as_view(), name='comentario_editar'),
17    path('comentario/<int:pk>/eliminar/', ComentarioDeleteView.as_view(), name='comentario_eliminar'),
18 ]
```

Con esto el camino que se seguirá es ingresar a la url de `categoria_list` desde el menú desplegable donde se encuentran todas las categorías. Una vez dentro, se elegirá una categoría donde estarán los posts de esa categoría, mediante la url `post_por_categoria.html` y en el listado de posts que vemos podremos acceder a un post en particular mediante la url que teníamos creada para `post_individual`. Como ves, esta es una forma de reutilizar vistas y urls creadas, y siempre se puede hacer de otras formas donde se puede reutilizar aún más o menos.

En `post_por_categoria.html` mostramos la lista de post ya filtrados y damos acceso a la vista del post individual.

```
templates > posts > post_por_categoria.html > ...
1 {% extends 'base.html' %}
2
3 {% block contenido %}
4 <center>
5 <div class="container-fluid" style="margin: 30px;">
6     <h1>Posts por categoria</h1>
7     <ul>
8         {% for post in posts %}
9             <li><a style="color: <input type="checkbox" checked="" href="{% url 'apps_posts:post_individual' id=post.id %}">{{ post.titulo }}</a></li>
10        </ul>
11    </div>
12 </center>
13 </block>
14 {% endblock %}
```

En `post_por_categoria.html` mostramos la lista de post ya filtrados y damos acceso a la vista del post individual. Probamos desde Colaborador1:



Como ves, ya tenemos enlazadas las rutas y accesos. Esto lo puedes integrar en un solo menú en tu proyecto o puedes hacerlo de otra forma. Las posibilidades son muchas.

Y ahora para finalizar, te mostraremos los últimos filtros que haremos para ordenar por fecha de más reciente a más antiguo de los posts, o, viceversa, y, por orden alfabético.

Para esto, modificaremos un poco nuestra vista de "PostListView" para lograr estos filtros:

```
apps > posts > views.py > ...
10 class PostListView(ListView):
11     model = Post
12     template_name = "posts/posts.html"
13     context_object_name = "posts"
14
15     def get_queryset(self):
16         queryset = super().get_queryset()
17         orden = self.request.GET.get('orden')
18         if orden == 'reciente':
19             queryset = queryset.order_by('-fecha')
20         elif orden == 'antiguo':
21             queryset = queryset.order_by('fecha')
22         elif orden == 'alfabetico':
23             queryset = queryset.order_by('titulo')
24         return queryset
25
26     def get_context_data(self, **kwargs):
27         context = super().get_context_data(**kwargs)
28         context['orden'] = self.request.GET.get('orden', 'reciente')
29         return context
```

Luego modificamos la plantilla posts.html para mostrar estas opciones:

```
templates > posts > posts.html > center > div.container-fluid > h2
1 {% extends 'base.html' %}
2 {% load static %}
3 {% load colaborador_tags %}
4
5 {% block contenido %}
6 <center>
7 <div class="container-fluid" style="margin: 300px;">
8
9 {% if user.is_superuser or request.user|has_group:"Colaborador" %}
10 <div class="container-fluid" style="margin-top: 300px;">
11 <a id="boton_post" href="{% url 'apps.posts:crear_post' %}">Crear nuevo post</a>
12 </div>
13 {% endif %}
14
15 <h2>Ordenar por:</h2>
16 <ul>
17 <a id="boton_post" href="{% url 'apps.posts:posts' %}?orden=reciente">Más reciente</a>
18 <a id="boton_post" href="{% url 'apps.posts:posts' %}?orden=antiguo">Más antiguo</a>
19 <a id="boton_post" href="{% url 'apps.posts:posts' %}?orden=alfabetico">Orden alfabético</a>
20 </ul>
21
22 {% for i in posts %}
23 <br>
24 <li>{{ i.titulo }}</li>
25 <li>{{ i.subtitulo }}</li>
26 <li>{{ i.categoria }}</li>
27 <br>
28 <a id="boton_post" href="{% url 'apps.posts:post_individual' i.id %}">
29 Ingresá a este Post
30 </a>
31 {% empty %}
32 <h1>No hay registros</h1>
33 {% endfor %}
34 </div>
35 </center>
36 {% endblock %}
```

Con esto al ingresar al los posts, podremos ordenar de cualquiera de estas formas. Te mostramos el resultado.

[Inicio](#) [Acerca de nosotros](#) [Posts](#)

[Crear nuevo post](#)

Ordenar por:

[Más reciente](#)

[Más antiguo](#)

[Orden alfabético](#)

- Post 1
- Subtitulo 1
- Categoría 2

[Ingresá a este Post](#)

- Post 2
- Subtitulo 2
- General

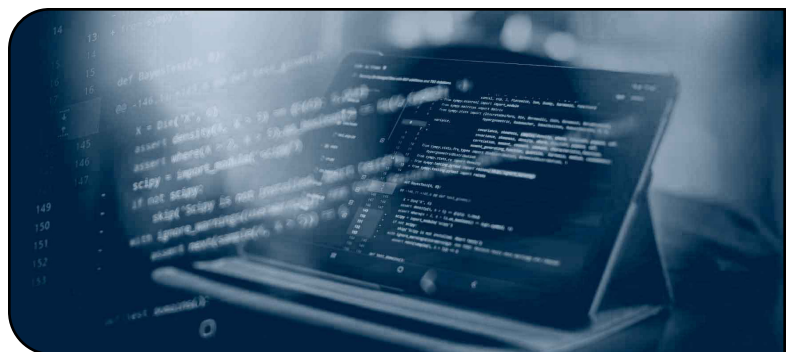
[Ingresá a este Post](#)

- Post 3
- Subtitulo 3
- Categoría 2

[Ingresá a este Post](#)

> Final

Finalización del manual



Con éstas bases, más lo que das en clases y la ayuda con las mentorías, ya tienes un buen conocimiento de Python, Django y SQL.

Recuerda que estas son solo bases para que puedas guiarte e implementar lo aprendido a tu propio proyecto. Siempre puede haber otras formas de resolver cada situación o consigna, y esperamos haberte ayudado con esta guía.

Sigue practicando y buscando diversas formas de resolver cada caso, siempre trata de ser lo más claro posible, busca siempre refactorizar el código para que cada vez sea más legible y fácil de entender. Recuerda que en la mayoría de los casos trabajarás con otros desarrolladores, por lo que mantener el orden, la legibilidad y la facilidad de lectura será lo fundamental para lograr el desarrollo de proyectos eficientes.

Busca ayuda en otros sitios, con otros desarrolladores, con tus profesores o colegas. Hay que reconocer las limitaciones de cada uno y tratar de superarse día a día, practicando y estudiando constantemente ya que esto no se logra de la noche a la mañana.

Desde aquí te deseamos muchos éxitos para tu proyecto final de esta segunda etapa del Informatario y esperamos que este manual te haya servido de ayuda.